

in response to the 'quit' command.

Figure 5 relates to the arrival of a new node in the existing network of nodes. New node X sends a *joinMe* request to the pre-existing node, which in turn publishes a *join* message to its neighbour nodes. All these nodes that receive a join message send their routing table to the newly joined node. New node X uses this information to populate its routing table and leaf set.

## Installation

The following steps will guide you in installing and running the application's node on any POSIX-compliant operating system. All of the code associated with the application is written in C++. Hence, a G++ compiler is required for successful installation of the application.

We have used protocol buffers to define a uniform interface for intercommunication between nodes. Install protocol buffers as described in the source repository at <https://github.com/protocolbuffers/protobuf/blob/master/src/README.md>.

For installation of *ProtoBuf*, the following tools are needed — *autoconf*, *automake*, *libtool*, *curl*, *make*, *g++*, *unzip*. Include the library for md5 in the same command used for installing all these tools. Run the following command to get all these tools installed on your system:

```
$ sudo apt-get install libssl-dev
autoconf automake libtool curl make g++
unzip
```

After completing the installation successfully, all these tools now proceed with installing *Protobuf*. You can get its source code for C++ by git cloning its Git repository as follows:

```
$ git clone https://github.com/
protocolbuffers/protobuf.git
$ cd protobuf
```

To make sure sub-modules have been cloned as well, run the following

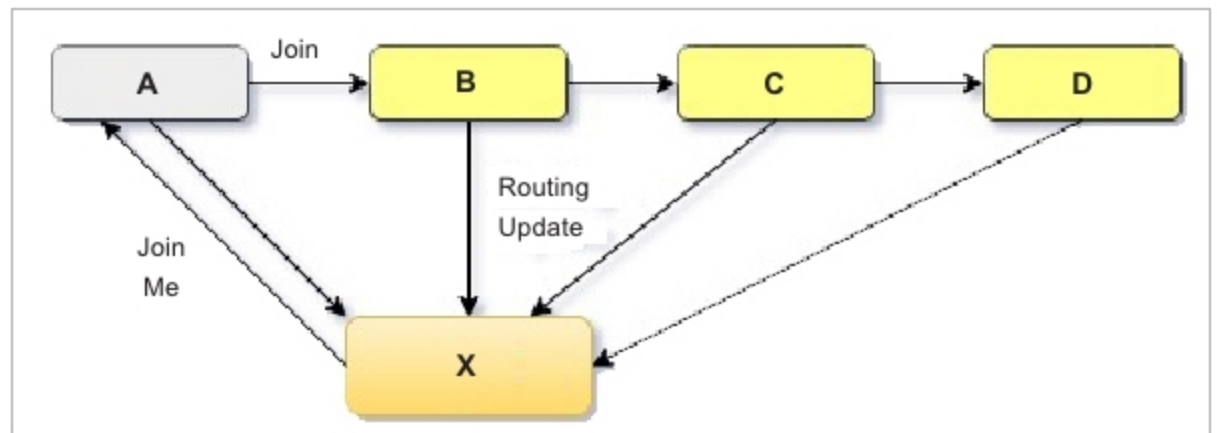


Figure 5: Arrival of a new node

command:

```
$ git submodule update --init
--recursive
```

Now generate the configuration scripts making use of *autoconf* and *automake* utilities, as follows:

```
$ ./autogen.sh
```

Make use of the *make* utility to build and install the C++ protocol buffer runtime and the protocol buffer compiler (protoc) by executing the following:

```
$ ./configure
$ make
$ sudo make install
$ sudo ldconfig # refresh shared library
cache
```

To install the application, get its source code from its Git repository at [https://github.com/murrdock/Key-Value\\_Store](https://github.com/murrdock/Key-Value_Store).

Execute the commands listed below to get the application's code:

```
$ git clone https://github.com/
murrdock/Key-Value_Store.git
```

Using the *make* utility, install the application by executing the commands as follows:

```
$ cd Key-Value_Store
$ make
```

After successfully compiling all files in the order defined inside *Makefile*, run the application's node as follows.

First, enter the *pastryClient* directory, where the executable from compilation is present.

```
$ cd pastryClient
```

Change the mode to superuser (root) for successful invocation of certain system calls in the code.

```
$ sudo su
```

```

arora@arora-Lenovo-Z51-70: ~/Desktop/Key-Value_Store/pastryClient
File Edit View Search Terminal Help
Key_Value_Store $ ./pastryClient LogMsgs
Welcome to Pasty client
You can Enter following commands
port <IP> <Port>      Initialise pastry
port <port>           It will take LANs IP
create               It will create a pastry listener on the above given commands
join <IP> <Port>      Give the ip and port of the pastry node through which you want to
join
put <Key> <Value>     It will store the value against the given key
get <Key>             It will retrieve the value stored against the key
nset                 Prints the Neighbourhood Set
lset                 Prints the Leaf Set
routetable           Prints Route Table
hashTable            Prints the Hash Table
quit                Gracefully closes the pastry node
shutdown            Shut Down the whole pastry Network
Enter Command...
```

Figure 6: Running instance of node